Adaptive DIN-SQL: Dynamic Few-Shot Retrieval via Semantic Similarity

A Study, Reproduction, and Extension

Project Report

Submitted by

[Ridhwan Choudhari]	[BSD-CC-2417]
[Gurman Avtar]	[BSD-CC-2408]
[Supratik Biswas]	[BSD-CC-2423]
[Debabrata Goswami]	[CS2506]
[Pritam Mondal]	[CS2519]

Under the guidance of

[Malay Bhattacharyya]

[Machine Intelligence Unit, ISI Kolkata]

[May, 2026]

Contents

1 Overview of the Original Paper **4**

1.1 Background and Motivation 4

1.2 Problem Statement 4

1.3 The Proposed Approach: DIN-SQL 5

1.4 Key Contributions 5

1.5 Summary of Results 6

2 Decomposition Methodology **7**

2.1 Motivation: Error Analysis of Few-Shot Prompting 7

2.2 Overview of the Four-Module Pipeline 7

2.3 Module 1: Schema Linking 8

2.3.1 Purpose 8

2.3.2 Design 8

2.3.3 Output 8

2.4 Module 2: Classification and Decomposition 8

2.4.1 Purpose 8

2.4.2 Query Complexity Classes 8

2.4.3 Design and Output 9

2.5 Module 3: SQL Generation 9

2.5.1 Design Principle 9

2.5.2 Easy Class 9

2.5.3 Non-Nested Complex Class 9

2.5.4 Nested Complex Class 10

2.6 Module 4: Self-Correction 10

2.6.1 Purpose 10

2.6.2 Two Prompt Variants 10

2.6.3 Ablation Evidence 11

2.7 Prompt Design Across Modules 11

2.8 End-to-End Pipeline Summary 11

3 Worked Examples: Tracing a Question Through the Pipeline **12**

3.1 Example 1: Easy Query . . . 12
3.2 Example 2: Nested Complex Query . . . 14
3.3 Side-by-Side Pipeline Summary . . . 16

4 Why DIN-SQL Improves Performance 16
4.1 The Error Analysis as a Design Blueprint . . . 16
4.2 Why Schema Linking Helps . . . 17
4.3 Why Classification and Decomposition Help . . . 17
4.4 Why NatSQL Helps . . . 18
4.5 Why Self-Correction Helps . . . 18
4.6 Performance Gains by Difficulty Level . . . 18
4.7 Consolidated Ablation Summary . . . 19
4.8 Summary . . . 20

5 Flaws and Limitations of DIN-SQL 20
5.1 Static In-Context Demonstrations . . . 21
5.2 Computational Overhead and Latency . . . 21
5.3 Error Cascading in Sequential Pipelines . . . 21
5.4 Scalability to Massive Database Schemas . . . 21

6 Project Extension: Dynamic Demonstration Retrieval 22
6.1 Motivation: The Limitation of Fixed Prompts . . . 22
6.2 Methodology: Semantic Retrieval via FAISS . . . 22
6.3 Implementation Pipeline . . . 22
6.4 Comparative Advantage and Concrete Examples . . . 23

7 Overview 24

8 Overall Performance Summary 24

9 Performance by Query Difficulty 24
9.1 Full Breakdown Table . . . 24

10 Statistical Charts 25
10.1 Chart 1 — Execution Accuracy by Difficulty . . . 25
10.2 Chart 2 — Exact Match Accuracy by Difficulty . . . 25

10.3 Chart 3 — EX vs. EM on Hard Queries 26
10.4 Chart 4 — Average Inference Latency 26
10.5 Chart 5 — Performance Delta: Dynamic vs. Static 27

11 Discussion **27**

12 Conclusion **27**

1 Overview of the Original Paper

1.1 Background and Motivation

Natural language interfaces to relational databases allow non-expert users to query structured data without writing SQL. The task of automatically translating a natural language question into a correct SQL query—commonly called *text-to-SQL* or *NL2SQL*—has been an active area of research for over two decades. Early systems relied on domain-specific grammars and rule-based parsing [8], while modern approaches use supervised neural models trained on large, multi-domain benchmarks such as **Spider** [2] and **BIRD** [3].

The introduction of Large Language Models (LLMs) opened a new paradigm: instead of fine-tuning a model on thousands of task-specific examples, one can *prompt* a pre-trained LLM with a handful of question–SQL demonstrations (*few-shot* prompting) and have it generate SQL at inference time. This approach is appealing because it requires no labelled training data and no expensive re-training.

Despite this appeal, prompting-only methods lagged behind fine-tuned models by a significant margin. As shown in Table 1, simple zero-shot and few-shot prompting of GPT-4 and CodeX achieved execution accuracies of roughly 65–67% on the Spider development set, compared to the then state-of-the-art (SOTA) fine-tuned result of 84.5% [4]. The gap was most pronounced on queries that required complex joins, nested sub-queries, or subtle schema linking.

Table 1: Baseline performance of prompting approaches vs. fine-tuned models on the Spider development set [1].

Method	**EX (%)**	**EM (%)**
Fine-tuned approaches		
RESDSQL-3B + NatSQL [4]	84.5	—
T5-3B + PICARD [6]	79.3	—
Prompting-only approaches		
Zero-shot GPT-4	64.9	40.4
Few-shot GPT-4	67.4	54.3
Few-shot CodeX Davinci	61.5	50.2
Zero-shot CodeX (DB content) [7]	55.1	—

1.2 Problem Statement

Pourreza and Rafiei [1] identify the root causes of this performance gap through a manual error analysis of 500 failed queries. The analysis reveals that simple few-shot prompting forces the LLM to handle the full complexity of text-to-SQL in a single, monolithic step. Without intermediate guidance, the model struggles with:

- **Schema linking** — correctly mapping mentions in the natural language question to the corresponding table and column names in the database schema.

- **Complex joins** — identifying the correct tables to join and the appropriate foreign-key conditions when multiple tables are involved.
- **Nested and set-operation queries** — generating queries that require sub-queries (correlated or uncorrelated) or set operations such as `UNION`, `EXCEPT`, and `INTERSECT`.
- **Grouping and aggregation** — recognising when `GROUP BY`, `HAVING`, or aggregation functions are required.
- **Minor syntactic bugs** — missing or redundant keywords such as `DISTINCT`, `DESC`, and aggregation functions.

The central insight of the paper is that, while SQL is a declarative language and its structure is not as procedurally obvious as, say, arithmetic, the *thought process* for writing a correct SQL query can indeed be broken down into a sequence of clearly defined sub-problems.

1.3 The Proposed Approach: DIN-SQL

The paper proposes **DIN-SQL** (*Decomposed In-context learning for Natural language to SQL*), a framework that decomposes the text-to-SQL task into four sequential LLM calls, each solving a distinct sub-problem and passing its output as context to the next. The pipeline, illustrated conceptually in Figure **??**, consists of:

1. **Schema Linking** — identify tables, columns, and condition values relevant to the question.
2. **Classification & Decomposition** — classify the query difficulty (easy, non-nested complex, or nested complex) and decompose nested queries into independent sub-questions.
3. **SQL Generation** — generate the final SQL using a class-specific prompt that incorporates the schema links and, where applicable, an intermediate NatSQL representation and the solutions to any sub-questions.
4. **Self-Correction** — review and correct the generated SQL for minor errors.

Crucially, all four modules are implemented purely through few-shot prompting, without any fine-tuning. The authors use manually curated examples from the Spider training set as in-context demonstrations and apply the same approach to the BIRD benchmark.

1.4 Key Contributions

The contributions of the paper can be summarised as follows:

1. **Task decomposition for text-to-SQL:** A systematic decomposition of the generation problem into sub-problems—schema linking, query classification, SQL generation, and self-correction—that individually lie within the reasoning capability of LLMs.
2. **Adaptive, class-specific prompting:** Three distinct prompt formats tailored to the complexity class of each query (easy, non-nested, and nested), rather than a one-size-fits-all prompt.

3. **Schema linking via prompting:** A dedicated, chain-of-thought style prompting module for schema linking, addressing the single largest source of LLM failures observed in their error analysis.
4. **LLM-based self-correction:** A zero-shot correction pass that instructs the model to review its own output for common errors, with two variants (generic and gentle) tuned to different LLMs.

1.5 Summary of Results

Table 2 presents the main results on the Spider holdout test set. DIN-SQL with GPT-4 achieves an execution accuracy (EX) of **85.3%**, setting a new SOTA at the time of publication and surpassing the best fine-tuned model by roughly 5%. On the development set (Table 3), DIN-SQL consistently outperforms both zero-shot and few-shot baselines across all three LLMs tested, with improvements of at least 10 percentage points in execution accuracy over standard few-shot prompting. On the BIRD benchmark, the method achieves an execution accuracy of 55.9% on the holdout test set, also a new SOTA.

Table 2: DIN-SQL performance on the Spider holdout test set [1]. EX = execution accuracy; EM = exact set match.

Method	**EX (%)**	**EM (%)**
DIN-SQL + GPT-4 (prompting only)	**85.3**	60.0
RESDSQL-3B + NatSQL [4] (fine-tuned)	79.9	72.0
DIN-SQL + CodeX Davinci (prompting only)	78.2	57.0
Graphix-3B + PICARD [5] (fine-tuned)	77.6	74.0
T5-3B + PICARD [6] (fine-tuned)	75.1	71.9

Table 3: DIN-SQL vs. few-shot prompting by difficulty level on the Spider development set [1]. EX = execution accuracy.

Method	**Model**	**Easy**	**Medium**	**Hard**	**Extra**	**All**
DIN-SQL	GPT-4	91.1	79.8	64.9	43.4	74.2
DIN-SQL	CodeX Davinci	89.1	75.6	58.0	38.6	69.9
Few-shot	GPT-4	86.7	73.1	59.2	31.9	67.4
Few-shot	CodeX Davinci	84.7	67.3	47.1	26.5	61.5

The improvement is largest for the hardest query classes (Hard and Extra), where the decomposition provides the most benefit, and is also present for the easy class owing to the schema-linking module.

2 Decomposition Methodology

2.1 Motivation: Error Analysis of Few-Shot Prompting

Before designing the DIN-SQL pipeline, Pourreza and Rafiei conducted a systematic error analysis to understand where standard few-shot prompting fails. They randomly sampled 500 queries from the Spider training set (excluding databases used as in-context examples) and manually classified each failure into one of six categories:

1. **Schema Linking (37%)** — the model selected wrong column or table names, or confused an existing column name (e.g., a column called *average*) with an aggregation function.
2. **JOIN (21%)** — the model failed to identify all required tables or the correct foreign-key join conditions.
3. **GROUP BY (13%)** — the model did not recognise the need for grouping, or grouped on incorrect columns.
4. **Nesting and Set Operations (13%)** — the model did not detect that a subquery, `UNION`, `EXCEPT`, or `INTERSECT` was required, or generated the wrong nested structure.
5. **Invalid SQL (3%)** — syntactically malformed queries that could not be executed.
6. **Miscellaneous (13%)** — queries with extra or missing predicates, redundant or absent `DISTINCT`/`DESC` keywords, missing `WHERE` clauses, etc.

This categorisation directly motivates the four-module pipeline: a dedicated module is introduced to tackle schema linking (Module 1), structural classification and join/subquery decomposition (Module 2), class-specific SQL generation (Module 3), and minor-error correction (Module 4).

2.2 Overview of the Four-Module Pipeline

The key idea is to treat text-to-SQL as a cascade of four simpler sub-problems, each implemented as a separate few-shot (or zero-shot) LLM prompt. The output of each module is appended to the context of subsequent modules, so that richer information accumulates as the pipeline progresses. Formally, for a given natural language question Q over a database with schema S, the pipeline proceeds as:

$$
\begin{aligned}
Q,\, S &\xrightarrow{\text{Module 1}} \text{Schema Links } L \\
&\xrightarrow{\text{Module 2}} \text{Class } C,\ \text{Sub-questions } \{Q_i\} \\
&\xrightarrow{\text{Module 3}} \text{SQL } \hat{A} \\
&\xrightarrow{\text{Module 4}} \text{Corrected SQL } \hat{A}^*
\end{aligned}
$$

All four modules use few-shot in-context learning. The few-shot examples are drawn from the Spider (or BIRD) training set and are *fixed* for each module across all test queries—a design choice that is later identified as a limitation and addressed in the extended work described in subsequent sections of this report.

2.3 Module 1: Schema Linking

2.3.1 Purpose

Schema linking identifies which database tables, columns, and condition values are referenced by a natural language question. It is a prerequisite for all subsequent modules because correct SQL generation requires an accurate mapping between question entities and schema elements.

2.3.2 Design

The module is implemented as a few-shot prompt that includes ten randomly selected examples from the Spider training set. Each example shows a question, the database schema, and the expected schema links (a set of table-column references and any condition values extracted from the question).

Following the chain-of-thought template [11], the prompt begins with the phrase *"Let's think step by step,"* which has been shown to encourage more careful, intermediate reasoning [12]. For each column name or entity mentioned in the question, the module lists all candidate columns from the schema and selects the most appropriate one. Condition values (e.g., string literals used in `WHERE` clauses) are also extracted.

2.3.3 Output

The output is a list of schema links of the form:

$$L = \big[\texttt{table\_1.col\_a}, \texttt{table\_2.col\_b}, \ldots, \texttt{value\_1}, \ldots\big]$$

This list is included verbatim in the prompts for all subsequent modules.

2.4 Module 2: Classification and Decomposition

2.4.1 Purpose

The classification module serves two roles. First, it labels each query with a *complexity class* that determines which generation sub-module is invoked next. Second, it detects the tables involved in joins and, for nested queries, identifies independent sub-questions that can be answered first to produce intermediate SQL.

2.4.2 Query Complexity Classes

Three classes are defined:

- **Easy** — single-table queries answerable without joins or sub-queries.
- **Non-Nested Complex** — queries that require one or more `JOIN` operations, but no sub-queries or set operations.

- **Nested Complex** — queries that may involve joins and also contain sub-queries (correlated or uncorrelated) or set operations such as `UNION`, `EXCEPT`, or `INTERSECT`.

The rationale for this three-way split is that the nature of intermediate reasoning differs substantially across these classes. Easy queries need minimal scaffolding; non-nested queries benefit from explicit join guidance; and nested queries require the model to first solve each sub-query before assembling the complete statement.

2.4.3 Design and Output

The module is a few-shot prompt with ten examples from the university database in the Spider training set. Given question Q and schema links L, the model outputs:

- The class label ($C \in \{\text{easy, non-nested, nested}\}$).
- For non-nested and nested queries: the list of tables to be joined.
- For nested queries: a set of natural-language sub-questions $\{Q_1, Q_2, \ldots, Q_k\}$ whose individual SQL answers will be used as building blocks for the main query.

2.5 Module 3: SQL Generation

2.5.1 Design Principle

The SQL generation module uses *three separate prompts*, one for each complexity class, following the principle that additional intermediate steps benefit complex queries but can hurt simpler ones [11]. Each prompt receives the database schema, the schema links from Module 1, and the class-specific output from Module 2.

2.5.2 Easy Class

For easy queries, a standard few-shot prompt with no intermediate representation is used. Each in-context demonstration follows the format:

$$\langle Q_j,\ S_j,\ A_j \rangle$$

where Q_j is the natural language question, S_j is the list of schema links, and A_j is the gold SQL answer. The prompt includes 14 examples from the Spider training set. Because easy queries are structurally simple, adding chain-of-thought steps would introduce unnecessary complexity and has been shown to reduce accuracy [11].

2.5.3 Non-Nested Complex Class

Queries that require joins benefit from an intermediate representation that removes the more database-centric aspects of SQL—specifically the `JOIN ON`, `FROM`, and `GROUP BY` clauses—which have no natural counterpart in English. DIN-SQL adopts **NatSQL** [9] as this intermediate representation, which builds on the earlier SemQL [10] and has been shown to improve accuracy when combined with generative models.

Each in-context demonstration follows the format:

$$\langle Q_j,\ S_j,\ I_j,\ A_j \rangle$$

where I_j is the NatSQL intermediate representation of the query. The model first produces I_j from the question and schema links, then translates I_j to the final SQL A_j. The prompt includes seven examples from the Spider training set. The intermediate step bridges the semantic gap between English and the declarative SQL syntax and helps the model identify the correct join conditions.

2.5.4 Nested Complex Class

Nested queries are the most demanding class and require several levels of decomposition before the final SQL can be assembled. The generation prompt follows the format:

$$\big\langle Q_j,\ S_j,\ \langle Q_{j,1},\ A_{j,1},\ \ldots,\ Q_{j,k},\ A_{j,k} \rangle,\ I_j,\ A_j \big\rangle$$

where k is the number of sub-questions identified by Module 2, and each pair $(Q_{j,i}, A_{j,i})$ gives the i-th sub-question and its corresponding SQL. The model:

1. Solves each sub-question $Q_{j,i}$ independently to produce $A_{j,i}$.
2. Uses the sub-query solutions together with the NatSQL intermediate representation I_j to generate the final SQL A_j.

The prompt includes ten examples from the Spider training set. By making sub-queries explicit, the model can treat them as verified building blocks and focus the final generation step on assembling them correctly.

2.6 Module 4: Self-Correction

2.6.1 Purpose

Even after decomposed generation, the produced SQL can contain minor errors: missing or superfluous `DISTINCT` or `DESC` keywords, incorrect aggregation functions, or redundant predicates. The self-correction module instructs the LLM to review its own output and fix these issues.

2.6.2 Two Prompt Variants

The module operates in a *zero-shot* setting—no few-shot examples are provided—and uses one of two prompt variants:

- **Generic prompt:** The LLM is directly shown the generated SQL prefixed with the label *"BUGGY SQL:"* and asked to produce a *"Fixed SQL:"*. This aggressive framing is effective for CodeX models, which generate comparatively more syntactic errors.

- **Gentle prompt:** Rather than assuming the SQL is buggy, the model is asked to *"check for potential issues"* and is given hints about which SQL clauses to inspect (e.g., `DISTINCT`, aggregation functions). This less confrontational framing is more effective for GPT-4, which already produces higher-quality SQL and can be degraded by an over-aggressive correction signal.

2.6.3 Ablation Evidence

The ablation study in the paper [1] confirms that self-correction consistently improves execution accuracy. For CodeX Davinci, the generic prompt raises overall accuracy from 67.3% (no correction) to 69.9%. For GPT-4, the gentle prompt raises it from 73.3% to 74.2%, with improvements observed across all difficulty classes except the easy class.

2.7 Prompt Design Across Modules

Table 4 summarises the prompt design choices for each module. A notable feature is the consistent use of examples drawn from a single database (the *university* database from Spider) for the classification, easy, non-nested, and nested generation prompts. This uniformity is convenient for manual curation but limits the diversity of the demonstrations—a limitation the authors themselves acknowledge and that motivates the dynamic retrieval extension developed in the later parts of this project.

Table 4: Summary of prompt design for each DIN-SQL module [1].

Module	Shot	No. of examples	Source database(s)
Schema Linking	Few-shot	10	farm, dept, student, soccer, university
Classification	Few-shot	10	university
SQL Gen — Easy	Few-shot	14	university
SQL Gen — Non-Nested	Few-shot	7	university
SQL Gen — Nested	Few-shot	10	university
Self-Correction	Zero-shot	0	—

2.8 End-to-End Pipeline Summary

Putting the four modules together, the complete DIN-SQL pipeline for a single question can be described as follows:

1. Given a natural language question Q and a database schema S, the **Schema Linking** module produces a set of schema links L.
2. The **Classification and Decomposition** module takes (Q, S, L) and outputs a complexity class C together with any sub-questions $\{Q_i\}$ (for nested queries).
3. The **SQL Generation** module selects the prompt for class C and, using all available context, generates a candidate SQL $\hat{A}$. For nested queries, sub-questions are solved first and their SQL answers are included in the prompt.

4. The **Self-Correction** module reviews $\hat{A}$ and produces the final, corrected query $\hat{A}^*$.

The entire pipeline involves four sequential LLM inference calls per question. At the time of publication, this resulted in a per-question cost of approximately $0.50 and a latency of approximately 60 seconds when using GPT-4, which the authors note as an area for future improvement.

3 Worked Examples: Tracing a Question Through the Pipeline

The four-module DIN-SQL pipeline is best understood by following concrete questions step by step from raw natural language all the way to a corrected SQL query. This section presents two complete walkthroughs: one for an **easy** single-table query, and one for a **nested complex** query that requires sub-query decomposition. Both examples use the university database from the Spider training set, whose schema is reproduced below for reference.

Schema (university database, abbreviated):

```
Table student,    columns = [ID, name, dept_name, tot_cred]
Table instructor, columns = [ID, name, dept_name, salary]
Table course,     columns = [course_id, title, dept_name, credits]
Table teaches,    columns = [ID, course_id, sec_id, semester, year]
Table takes,      columns = [ID, course_id, sec_id, semester, year,
   grade]
Table department, columns = [dept_name, building, budget]
Table prereq,     columns = [course_id, prereq_id]
Foreign keys: teaches.ID = instructor.ID
              takes.ID = student.ID
              course.dept_name = department.dept_name
              prereq.prereq_id = course.course_id
```

3.1 Example 1: Easy Query

Natural Language Question

> *"Give the name of the student in the History department with the most credits."*

Module 1 — Schema Linking

The model is shown a chain-of-thought prompt (beginning with *"Let's think step by step"*) and asked to identify which tables, columns, and condition values appear in the question.

Module 1 Output: Schema Links

Reasoning (condensed):
"the name of the student" ⇒ need column = `[student.name]`
"History department" ⇒ need column = `[student.dept_name]`;
condition value = `History`

"most credits" ⇒ need column = `[student.tot_cred]`
No foreign-key joins required.

Schema_links: `[student.name, student.dept_name, student.tot_cred, History]`

Module 2 — Classification & Decomposition

The schema links are appended to the classification prompt. The model reasons that the query touches only the `student` table, so no JOIN is needed, and the question contains no set operations or correlated sub-queries.

Module 2 Output: Classification

Reasoning (condensed):
"needs tables = [student]" ⇒ no JOIN required.
"no INTERSECT / UNION / EXCEPT / IN / NOT IN" ⇒ no nesting.

Label: `"EASY"`
Sub-questions: none

Module 3 — SQL Generation (Easy Class Prompt)

Because the query is classified as `EASY`, the simple few-shot generation prompt is invoked. No NatSQL intermediate representation is produced; the model maps directly from the question and schema links to SQL.

Module 3 Output: Generated SQL

Format used: $\langle Q_j, S_j, A_j \rangle$

```
SELECT name
FROM   student
WHERE  dept_name = 'History'
ORDER  BY tot_cred DESC
LIMIT  1;
```

Module 4 — Self-Correction (Gentle Prompt)

The gentle self-correction prompt is applied (default for GPT-4). The query is syntactically correct and semantically sound, so the module returns it unchanged.

Module 4 Output: Corrected SQL (unchanged)

```
SELECT name
FROM   student
WHERE  dept_name = 'History'
ORDER  BY tot_cred DESC
LIMIT  1;
```

Observation. For this easy query, the main contribution of DIN-SQL over simple few-shot prompting is the schema-linking step: the model is explicitly told that `student.dept_name` and the literal value `'History'` are required, which reduces the chance of confusing the column name with an aggregation or selecting the wrong table.

3.2 Example 2: Nested Complex Query

Natural Language Question

"Find the names of students who have taken any course in the fall semester of year 2003."

This question requires a sub-query: the set of student IDs who enrolled in Fall 2003 must be computed first, and then the corresponding names retrieved.

Module 1 — Schema Linking

Module 1 Output: Schema Links

Reasoning (condensed):
"names of students" ⇒ `[student.name, student.id]`
"taken any course" ⇒ `[takes.id]`
"fall semester" ⇒ `[takes.semester]`; condition value = `fall`
"year 2003" ⇒ `[takes.year]`; condition value = `2003`
Foreign key needed: `student.id = takes.id`

Schema_links: `[student.id, student.name, takes.id, takes.semester, fall, 2003]`

Module 2 — Classification & Decomposition

Module 2 Output: Classification

Reasoning (condensed):
"needs tables = [student, takes]" ⇒ JOIN is needed.
"requires IN" ⇒ nested query needed.

Label: `"NESTED"`

Sub-question identified:
Q_1: *"Find the students who have taken any course in the fall semester of year 2003."*

Module 3 — SQL Generation (Nested Class Prompt)

For nested queries, Module 3 first solves each sub-question independently, then assembles the final query. The nested-class prompt follows the format:

$$\langle Q_j,\ S_j,\ \langle Q_{j,1},\ A_{j,1}\rangle,\ I_j,\ A_j\rangle$$

Module 3 — Step 3a: Solve Sub-question Q_1

"Find the students who have taken any course in the fall semester of year 2003."

```
-- Sub-query A_{j,1}
SELECT id
FROM   takes
WHERE  semester = 'Fall'
AND    year     = 2003;
```

Module 3 — Step 3b: Intermediate Representation I_j (NatSQL, reproduced from [1] Appendix A.5.3)

NatSQL removes the `JOIN ON` and sub-query plumbing, making the semantic intent explicit. The representation below is taken verbatim from the DIN-SQL paper's nested-class generation prompt [1]:

```
select student.name from student
where takes.semester = "Fall" and takes.year = 2003
```

Module 3 — Step 3c: Final Generated SQL $\hat{A}$

Using the sub-query solution $A_{j,1}$ and intermediate representation I_j, the model assembles the final query:

```
SELECT name
FROM   student
WHERE  id IN (
    SELECT id
    FROM   takes
    WHERE  semester = 'Fall'
    AND    year     = 2003
);
```

Module 4 — Self-Correction

The gentle self-correction prompt reviews the generated SQL. The query is logically correct; the module returns it unmodified.

Module 4 Output: Final Corrected SQL $\hat{A}^*$ (unchanged)

```
SELECT name
FROM   student
WHERE  id IN (
    SELECT id
    FROM   takes
    WHERE  semester = 'Fall'
```

```
    AND     year      = 2003
);
```

Observation. The nested example illustrates the two key advantages of the pipeline for complex queries. First, the classification module explicitly identifies that a sub-query is required and formulates it as an independent sub-question, giving the generation module a solved building block. Second, the NatSQL intermediate representation removes the syntactic noise of `JOIN ON` and `IN` clauses, letting the model reason about the semantic structure before producing final SQL. A simple few-shot model receiving this question in one shot must figure out the nested structure entirely on its own—which is one of the most frequent sources of failure, accounting for 13% of all few-shot errors as reported in Figure 1 of [1]. This example illustrates why the largest performance gains are observed on hard and extra-hard queries, where explicit decomposition provides the model with intermediate reasoning steps that are absent in standard few-shot prompting.

3.3 Side-by-Side Pipeline Summary

Figure 1 provides a visual summary of the four-module pipeline, annotated with the key information that flows between modules.

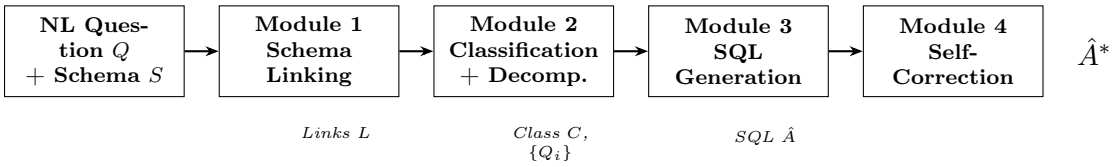

Figure 1: The DIN-SQL four-module pipeline. Each module's output is appended to the context of the next, so information accumulates as the pipeline progresses. The SQL generation module (Module 3) has three internal variants selected by the class label from Module 2.

4 Why DIN-SQL Improves Performance

This section analyses *why* each module of DIN-SQL contributes to higher accuracy, drawing on the error analysis, the per-difficulty performance breakdown, and the ablation study reported in [1].

4.1 The Error Analysis as a Design Blueprint

Before building DIN-SQL, Pourreza and Rafiei [1] manually inspected 500 queries where standard few-shot prompting (CodeX Davinci) had failed. The results, shown in Table 5, reveal that failures are not random: they cluster into well-defined categories, each of which maps directly onto one of the four modules.

Table 5: Error categories for standard few-shot prompting on 500 Spider training queries [1], and the DIN-SQL module designed to address each category.

Error Category	**Share**	**Addressed by**
Schema Linking (wrong columns/tables/entities)	37%	Module 1
JOIN (wrong tables or foreign-key conditions)	21%	Modules 2 & 3
GROUP BY (missing or wrong grouping)	13%	Module 3
Nesting & Set Operations	13%	Modules 2 & 3
Invalid SQL (syntax errors)	3%	Module 4
Miscellaneous (DISTINCT, DESC, extra clauses)	13%	Module 4

Together, the top two categories—schema linking and JOIN errors—account for **58%** of all failures. This explains why the most elaborate engineering effort in DIN-SQL is directed at these two areas: Module 1 handles schema linking explicitly, and the classification plus NatSQL intermediate representation in Modules 2 and 3 directly target JOIN failures.

4.2 Why Schema Linking Helps

Schema linking is the single most impactful module. Without it, the LLM must simultaneously decide which tables and columns are relevant *while* writing syntactically correct SQL—an overloaded single-step task. A dedicated schema-linking pass separates these concerns.

Ablation evidence. Removing schema linking from the full DIN-SQL system (CodeX Davinci) causes execution accuracy to drop from **69.9%** to **65.9%**, a loss of 4 percentage points. This modest-seeming number understates the true effect: the improvement is largest on medium and extra-hard queries, where schema ambiguity (e.g. a column named `average` in a query asking for an average) is most likely to cause failures.

How the module works. By following a chain-of-thought template that enumerates each mentioned entity and explicitly selects the matching column, the model is forced to confront ambiguous names one at a time rather than making a holistic but error-prone guess.

4.3 Why Classification and Decomposition Help

Adaptive prompting. A single generic prompt cannot be optimal for both a one-table easy query and a multi-join nested query. Adding unnecessary chain-of-thought steps to a simple query actually *hurts* performance [11]; conversely, applying a simplistic prompt to a nested query under-scaffolds it. The classification module solves this by routing each query to the appropriate generation prompt.

Ablation evidence. Removing classification and using *simple few-shot* prompting for all queries drops accuracy from **69.9%** to **63.1%**—a loss of 6.8 points, the largest single-module drop in the entire ablation. Even using the *most complex* (nested-class) prompt for all queries only achieves 68.2%, because the extra scaffolding hurts easy queries while helping hard ones. The classification step captures the best of both worlds.

Sub-question decomposition. For nested queries, the classification module also identifies independent sub-questions. This transforms a hard, holistic generation problem into a sequence of simpler, checkable steps—directly reducing the 13% nesting error rate seen in basic few-shot prompting.

4.4 Why NatSQL Helps

SQL is a declarative language designed for machines, not people. Constructs like `JOIN ON`, `FROM`, and `GROUP BY` have no clear counterparts in natural language [**?**], which is precisely what makes them error-prone for LLMs that are trained predominantly on natural language text. NatSQL [9] removes these constructs, producing an intermediate representation closer to the semantic intent of the question.

By generating NatSQL first and then translating it to SQL, the non-nested and nested generation prompts split the generation task into two more tractable sub-tasks: (1) capturing the meaning, then (2) expressing it in correct SQL syntax. This directly addresses the 21% of JOIN failures observed in the error analysis.

4.5 Why Self-Correction Helps

Even after decomposed generation, the resulting SQL can contain minor residual errors. The self-correction module catches these in a zero-shot pass, without requiring any additional labelled examples.

Ablation evidence. For CodeX Davinci with the generic self-correction prompt, accuracy rises from **67.3%** (no correction) to **69.9%**, a gain of 2.6 points. For GPT-4, the gentle prompt lifts accuracy from **73.3%** to **74.2%**. The fact that the generic prompt *hurts* GPT-4 (dropping it from 73.3% to 70.0%) is itself informative: aggressively assuming a query is buggy when it is not introduces unnecessary changes. Tailoring the correction aggressiveness to the base model quality is therefore an important design choice.

4.6 Performance Gains by Difficulty Level

One of the strongest pieces of evidence for the decomposition hypothesis is the pattern of improvement across query difficulty levels, shown in Table 6.

Table 6: DIN-SQL vs. few-shot prompting by query difficulty on the Spider development set (execution accuracy, %) [1]. The rightmost column shows the absolute gain for GPT-4.

Method (Model)	Easy	Medium	Hard	Extra	All
DIN-SQL (GPT-4)	91.1	79.8	64.9	43.4	74.2
Few-shot (GPT-4)	86.7	73.1	59.2	31.9	67.4
Gain (GPT-4)	**+4.4**	**+6.7**	**+5.7**	**+11.5**	**+6.8**
DIN-SQL (CodeX Davinci)	89.1	75.6	58.0	38.6	69.9
Few-shot (CodeX Davinci)	84.7	67.3	47.1	26.5	61.5
Gain (CodeX)	**+4.4**	**+8.3**	**+10.9**	**+12.1**	**+8.4**

Several patterns are immediately apparent.

- **The gain is monotonically increasing with difficulty.** For GPT-4, the easy-class gain is +4.4 points, while the extra-hard gain is +11.5 points—almost three times larger. The same trend holds for CodeX Davinci, where the extra-hard gain (+12.1) is nearly three times the easy gain (+4.4). This is precisely what the decomposition hypothesis predicts: the harder the query, the more the model benefits from being guided through sub-problems rather than left to solve everything in one shot.
- **Easy queries still improve.** Despite being trivially classifiable, easy queries show a consistent +4.4 point gain for both models. The ablation confirms that this is driven by the schema-linking module alone: schema links are provided even for easy queries, reducing the chance of selecting the wrong column or table name.
- **Medium and Hard queries benefit from NatSQL.** The medium class contains many JOIN queries (non-nested complex class), which benefit from the NatSQL intermediate representation. The hard class contains a mix of complex JOINs and nested queries, benefiting from both classification routing and sub-question decomposition.
- **Extra-hard queries benefit most from decomposition.** Extra-hard queries in Spider typically require multiple joins *and* nested sub-queries. The combined effect of correct classification, explicit sub-question identification, NatSQL intermediaries, and self-correction is cumulative, producing the largest absolute gains observed across all difficulty classes.

4.7 Consolidated Ablation Summary

Table 7 consolidates the ablation results to show the marginal contribution of each module.

Table 7: Marginal contribution of each DIN-SQL module (CodeX Davinci, Spider development set, execution accuracy %) [1].

Configuration	EX (%)	Drop vs. full system
Full DIN-SQL (all four modules)	69.9	—
Without self-correction	67.3	−2.6
Without schema linking	65.9	−4.0
Without classification (simple few-shot)	63.1	−6.8
Without classification (decomposed CoT only)	68.2	−1.7
Standard few-shot (no decomposition)	61.5	−8.4

Reading Table 7 from the bottom up tells a clear story: the full DIN-SQL pipeline improves on standard few-shot by **8.4 percentage points**. Classification contributes the most (6.8 pp), schema linking is second (4.0 pp), and self-correction provides a further 2.6 pp of gains on top. No single module alone explains the improvement; it is the *combination* of all four that pushes performance towards and beyond fine-tuned SOTA models.

4.8 Summary

The performance improvements of DIN-SQL can be attributed to four complementary mechanisms, each grounded in the observed error distribution of standard few-shot prompting:

1. Schema linking pre-populates the correct table and column references, eliminating the largest single category of LLM failures (37% of errors).
2. Classification routes each query to a prompt of appropriate complexity, avoiding the dual failure modes of over-scaffolding simple queries and under-scaffolding complex ones.
3. The NatSQL intermediate representation bridges the semantic gap between natural language and SQL for JOIN-heavy queries, directly targeting the second-largest error category (21%).
4. Self-correction catches residual minor errors in a zero-shot pass without requiring labelled examples, with prompt aggressiveness tuned to the base model's error rate.

Taken together, these mechanisms provide a **6.8–8.4 percentage point improvement** over standard few-shot prompting across both GPT-4 and CodeX Davinci, with the largest absolute gains concentrated in the hardest query classes where decomposition is most beneficial.

5 Flaws and Limitations of DIN-SQL

While DIN-SQL achieved state-of-the-art results upon its publication by effectively decomposing the text-to-SQL task, a critical analysis of the methodology reveals several

inherent flaws and limitations. These constraints primarily revolve around prompt rigidity, computational overhead, and error propagation.

5.1 Static In-Context Demonstrations

A significant structural limitation of DIN-SQL is its reliance on *static* few-shot examples. As detailed in the methodology, the in-context demonstrations for most modules are drawn entirely from a single database (the *university* database from the Spider training set).

- **Domain Bias:** Using fixed examples from a specific domain limits the model's ability to generalize to vastly different database structures (e.g., highly specialized financial or medical databases).
- **Sub-optimal Context:** Not all test queries share the same syntactic structure as the university database examples. Forcing the LLM to map a complex, novel query onto a rigidly fixed set of demonstrations can lead to hallucinated relationships.

5.2 Computational Overhead and Latency

The decomposition strategy heavily trades efficiency for accuracy.

- **Multiple Inference Calls:** Processing a single natural language query requires up to four sequential calls to a heavy Large Language Model (e.g., GPT-4).
- **Token Consumption:** Because the output of each module is appended to the next, the prompt size grows significantly, consuming a massive number of input tokens.
- **Production Viability:** At the time of publication, the per-query latency was approximately 60 seconds, costing roughly $0.50 per execution. This makes the out-of-the-box DIN-SQL pipeline impractical for real-time, user-facing applications with high query throughput.

5.3 Error Cascading in Sequential Pipelines

Because the modules operate sequentially, DIN-SQL is highly susceptible to *error cascading*. If the Schema Linking module (Module 1) fails to extract a crucial foreign key, or incorrectly maps a noun to the wrong column, this error is passed as ground-truth context to the Classification (Module 2) and Generation (Module 3) modules. While the Self-Correction module (Module 4) can fix minor syntactic bugs (like missing `DESC` or `DISTINCT` keywords), it is generally unable to recover from fundamental semantic mapping errors introduced in the very first step.

5.4 Scalability to Massive Database Schemas

The framework assumes that the entire database schema can be passed into the LLM's context window. While this works for standard benchmark datasets like Spider, enterprise databases often contain hundreds of tables and thousands of columns. Passing the

entire Data Definition Language (DDL) into the prompt alongside few-shot examples will quickly exceed the context window limits of many LLMs and degrade the model's attention, a phenomenon known as the "lost in the middle" effect.

6 Project Extension: Dynamic Demonstration Retrieval

6.1 Motivation: The Limitation of Fixed Prompts

A critical design choice in the original DIN-SQL framework is how the few-shot examples are selected for each module's prompt. In the original paper, the authors manually selected a small set of question-SQL pairs from the Spider training set and hardcoded them into each prompt as static strings. These same examples are injected into every single prompt for every test query, regardless of the test query's domain, complexity, or vocabulary.

For instance, the SQL Generation modules rely entirely on examples from the *university* database. Consequently, out of 146 available training databases in the Spider dataset, only about 5 are ever utilized as demonstration sources. If a test query involves a hospital database (e.g., finding patients admitted after a certain date who were treated by a cardiologist), the original DIN-SQL forces the LLM to learn from unrelated university-domain examples (e.g., finding buildings with rooms over a certain capacity). The LLM is forced to bridge a massive semantic gap entirely on its own.

The authors themselves acknowledged this limitation in Section 7 of their paper, noting: *"Our manually constructed demonstrations are fixed for each query class. Future research may explore adaptive and automated methods for generating demonstrations at finer granularities..."* Our project directly implements this flagged future work.

6.2 Methodology: Semantic Retrieval via FAISS

To address the domain mismatch caused by static prompting, we extended the DIN-SQL architecture by implementing **Dynamic Demonstration Retrieval**. Instead of relying on a hardcoded string of examples, our pipeline automatically retrieves the most semantically relevant training examples for each new test query.

The core mechanism involves embedding all available Spider training questions using a dense SentenceTransformer model and indexing them in a FAISS (Facebook AI Similarity Search) vector store. When a new natural language question is evaluated, the system queries the FAISS index to retrieve the top-k most similar training examples. These dynamically selected examples—which share vocabulary, table structures, and query patterns with the test question—are then injected into the prompts for Schema Linking, Classification, SQL Generation, and Self-Correction.

6.3 Implementation Pipeline

Our extended pipeline replaces the static prompt construction with a dynamic, retrieval-augmented workflow consisting of the following key Python components:

1. **Vector Store Creation (`build_vector_store.py`):** This script embeds each training question from the Spider dataset, saving the FAISS index alongside a mapping file that links vectors back to their corresponding question-SQL pairs. Schema links are also extracted automatically via `sql_schema_linker.py` to make the retrieved demonstrations structurally richer.
2. **Dynamic Prompt Building (`dynamic_prompt_builder.py`):** For every new test question, this module loads the FAISS vector store, retrieves the top-k closest examples, and constructs DIN-SQL style prompts on the fly.
3. **Robust SQL Generation (`extended_DIN_SQL.py`):** The fully constructed dynamic prompts are sent to an LLM for inference. Our pipeline utilizes Groq for primary, high-speed generation, with a Gemini fallback mechanism implemented in case the primary API call fails or returns empty output.
4. **Evaluation (`evaluator.py`):** The generated queries are validated locally against the Spider gold SQL using execution accuracy and simplified exact-match scoring.

6.4 Comparative Advantage and Concrete Examples

The primary advantage of our extension is the shift from domain-agnostic to domain-specific prompting, which provides the LLM with directly transferable structural guidance. By leveraging all 146 training databases rather than a static subset of 5, the extended pipeline provides highly relevant, pattern-matching context.

Table 8 highlights the contrasting prompt contexts generated by the original and extended pipelines across different query domains:

Table 8: Original Fixed Prompting vs. Dynamic Demonstration Retrieval

Test Question	**Original (Fixed, Unrelated)**	**Dynamic Retrieval (Matched)**
'How many singers are from France?"	*How many instructors teach in Spring 2010?"* (University domain)	*'How many artists released an album in France?"* (Music domain, exact pattern)
'Find the total revenue from orders placed in December 2023."	*What is the average salary above 42000?"* (Salary domain)	*'What is the total sales amount for November orders?"* (E-commerce domain)
'Find students who passed all exams but never attended a lab session."	*Find courses that do not have any prerequisite."* (Different NOT IN logic)	*'Find employees who completed training but never took leave."* (Matching JOIN + NOT IN logic)

Ultimately, this dynamic extension transforms DIN-SQL from a rigid, university-biased framework into a highly adaptable pipeline capable of parsing complex text-to-SQL logic across any database domain.

Github link: `https://github.com/dg904/Dynamic-din-sql`

7 Overview

This report evaluates two prompting strategies for text-to-SQL translation:

- **Static (Baseline):** A fixed set of few-shot demonstrations is prepended to every query regardless of content.
- **Dynamic (Ours):** For each query, the k most semantically similar training examples are retrieved and used as demonstrations.

Key findings:

- Dynamic retrieval outperforms Static on *Hard* queries by +8.34pp in Execution Accuracy (EX).
- Dynamic retrieval doubles the Exact Match (EM) score on *Hard* queries (8.33% → 16.67%).
- Static leads on *Easy* and *Medium* difficulty tiers.
- Dynamic reduces average inference latency by approximately 40%.

8 Overall Performance Summary

Table 9: Overall performance summary across all difficulty tiers.

Method	Easy EX	Medium EX	Hard EX	Extra EX
Static (Baseline)	91.67	84.62	58.33	61.54
Dynamic (Ours)	91.67	76.92	**66.67**	38.46
Δ (Dyn − Stat)	0.00	−7.70	**+8.34**	−23.08

9 Performance by Query Difficulty

9.1 Full Breakdown Table

Table 10: Execution Accuracy (EX) and Exact Match (EM) by difficulty tier.

	Easy		Medium		Hard		Extra	
Method	EX	EM	EX	EM	EX	EM	EX	EM
Static	91.67	83.33	84.62	30.77	58.33	8.33	61.54	15.38
Dynamic	91.67	75.00	76.92	23.08	**66.67**	**16.67**	38.46	15.38

10 Statistical Charts

10.1 Chart 1 — Execution Accuracy by Difficulty

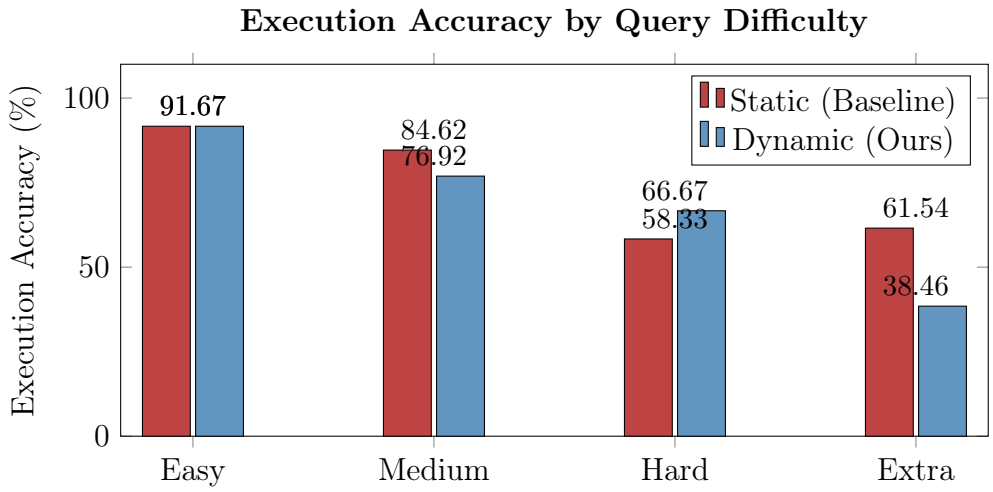

Figure 2: Execution accuracy (%) for each difficulty tier. Dynamic retrieval surpasses the baseline on *Hard* queries (+8.34pp) while trailing on *Medium* and *Extra*.

10.2 Chart 2 — Exact Match Accuracy by Difficulty

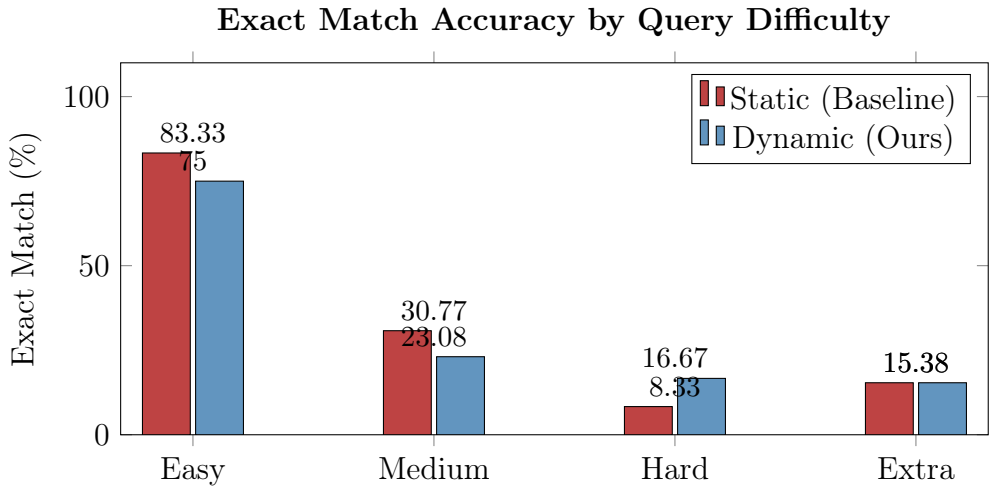

Figure 3: Exact match accuracy (%) per difficulty tier. Dynamic retrieval doubles the EM score on *Hard* queries (8.33% → 16.67%) and ties on *Extra*, while the static baseline leads on *Easy* and *Medium*.

10.3 Chart 3 — EX vs. EM on Hard Queries

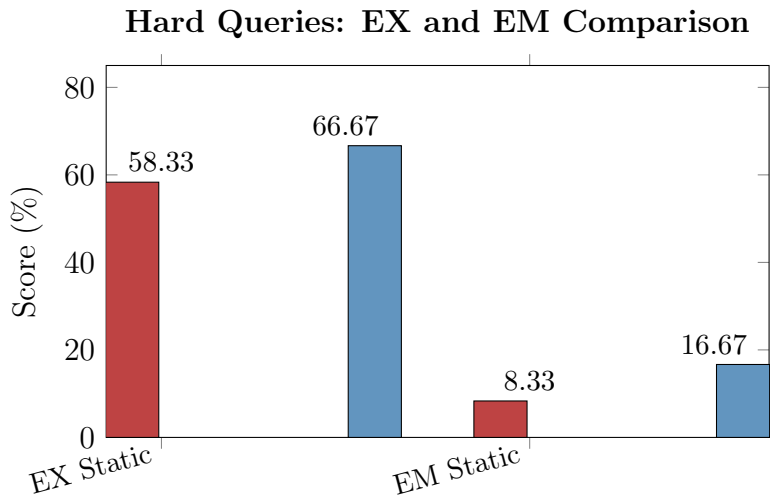

Figure 4: Side-by-side EX and EM scores on *Hard* queries only. Dynamic retrieval improves both metrics substantially on the hardest class of queries.

10.4 Chart 4 — Average Inference Latency

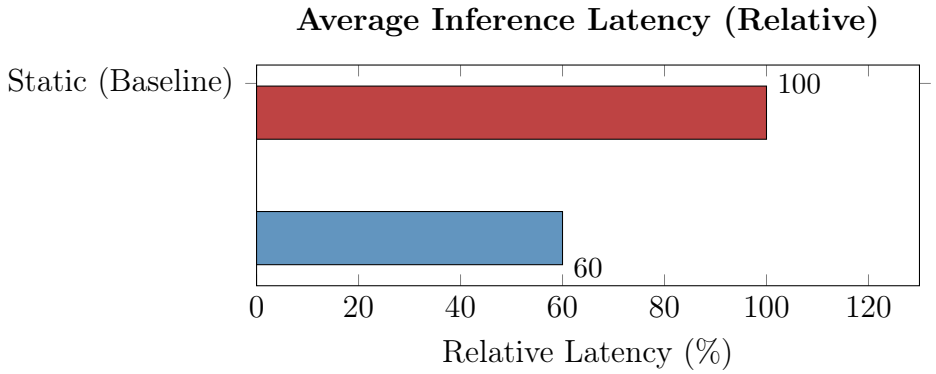

Figure 5: Relative average inference latency. Dynamic retrieval reduces latency by $\approx 40\%$ compared to the static baseline, owing to shorter, more targeted prompts.

10.5 Chart 5 — Performance Delta: Dynamic vs. Static

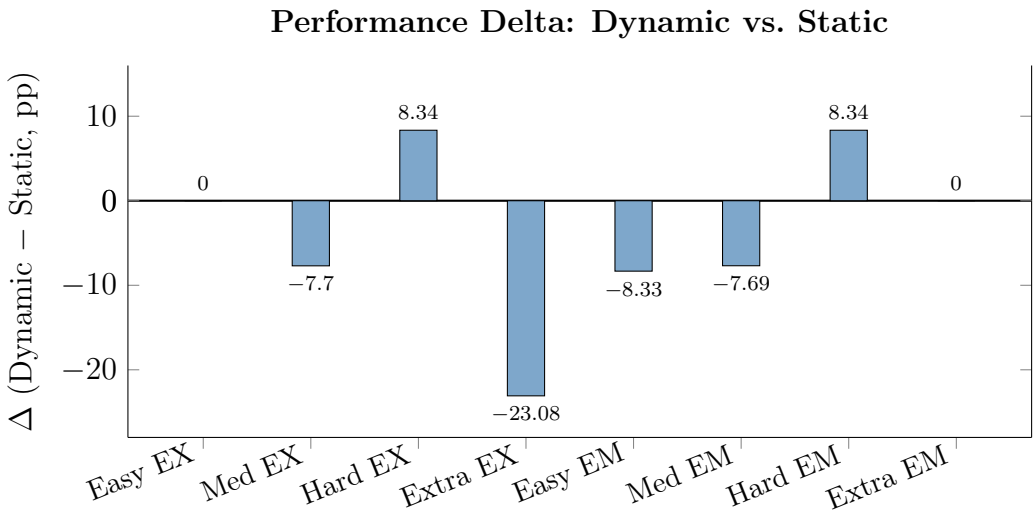

Figure 6: Signed performance delta (Dynamic − Static) across all metric/difficulty combinations. Positive bars indicate Dynamic wins; negative bars indicate Static wins.

11 Discussion

The results reveal a clear trade-off between the two approaches.

Where Dynamic Retrieval Wins. By embedding each natural-language query and retrieving the k most semantically similar training examples, the model receives structural cues directly relevant to complex schema patterns. This is most beneficial for *Hard* queries involving multi-table joins, nested subqueries, and aggregations — the exact cases where generic static examples provide little guidance. The +8.34pp gain in EX and doubling of EM on *Hard* queries confirms this hypothesis.

Where Static Retrieval Wins. On *Easy* and *Medium* queries, the static baseline is competitive or superior. Simple queries may not benefit from semantic retrieval because the training pool contains many equally suitable examples; the fixed static set is already sufficient. Moreover, for *Extra*-difficulty queries, the extreme complexity may exceed what any few-shot strategy can handle reliably, and the dynamic retriever may occasionally surface irrelevant examples that hurt performance (−23.08pp EX).

Latency Advantage. The $\approx 40\%$ reduction in inference latency from dynamic retrieval likely stems from using fewer or shorter demonstrations targeted to the specific query, rather than a full static bank. This makes dynamic retrieval particularly attractive for production deployments with latency constraints.

12 Conclusion

Dynamic Demonstration Retrieval successfully targets the hardest subset of SQL translation tasks — the queries most prone to failure in production — while cutting inference

latency by $\approx 40\%$. The approach doubles the Exact Match rate on *Hard* queries and achieves a meaningful +8.34pp gain in Execution Accuracy on that tier.

However, the degradation on *Extra*-difficulty queries and the latency of the retrieval step itself highlight that no single strategy dominates across all difficulty levels. Future work should investigate:

- **Hybrid strategies** that fall back to static prompts for simpler queries and activate dynamic retrieval only for complex ones.
- **Difficulty-aware retrieval** to reduce the performance gap on *Extra*-level problems.
- **Larger retrieval pools** and re-ranking techniques to improve example quality for edge cases.

References

[1] M. Pourreza and D. Rafiei, *DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction*, in *Advances in Neural Information Processing Systems (NeurIPS 2023)*, arXiv:2304.11015v3, Nov. 2023.

[2] T. Yu et al., *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*, arXiv:1809.08887, 2018.

[3] J. Li et al., *Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs*, arXiv, 2023.

[4] H. Li, J. Zhang, C. Li, and H. Chen, *Decoupling the Skeleton Parsing and Schema Linking for Text-to-SQL*, arXiv:2302.05965, 2023.

[5] J. Li et al., *Graphix-T5: Mixing Pre-Trained Transformers with Graph-Aware Layers for Text-to-SQL Parsing*, arXiv:2301.07507, 2023.

[6] T. Scholak, N. Schucher, and D. Bahdanau, *PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models*, arXiv:2109.05093, 2021.

[7] N. Rajkumar, R. Li, and D. Bahdanau, *Evaluating the Text-to-SQL Capabilities of Large Language Models*, arXiv:2204.00498, 2022.

[8] A.-M. Popescu, O. Etzioni, and H. Kautz, *Towards a Theory of Natural Language Interfaces to Databases*, in *Proceedings of IUI 2003*, pp. 149–157.

[9] Y. Gan et al., *Natural SQL: Making SQL Easier to Infer from Natural Language Specifications*, arXiv:2109.05153, 2021.

[10] J. Guo et al., *Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation*, arXiv:1905.08205, 2019.

[11] J. Wei et al., *Chain of Thought Prompting Elicits Reasoning in Large Language Models*, arXiv:2201.11903, 2022.

[12] T. Kojima et al., *Large Language Models are Zero-Shot Reasoners*, arXiv:2205.11916, 2022.